

Java-Referenz

Benjamin Malicke ©

May 12, 2026

Contents

1	Einführung	2
2	Java-Schlüsselwörter	3
3	Java String-Methoden	5
4	Java Math-Methoden	9
5	Java Ausgabe-Methoden	13
6	Java Arrays-Methoden	14
7	Java ArrayList-Methoden	15
8	Java LinkedList-Methoden	17
9	Java HashMap-Methoden	20
10	Java Scanner-Methoden	22
11	Java File-Methoden	24
12	Java FileInputStream-Methoden	26
13	Java FileOutputStream-Methoden	27
14	Java BufferedReader-Methoden	28
15	Java BufferedWriter-Methoden	29
16	Java Iterator-Methoden	30
17	Java Collections-Methoden	31
18	Java System-Methoden	32
19	Java Fehler und Ausnahmen	33
20	Java-Datum	33
21	Java-Typenumwandlungen	39
22	Java-Parsen	40
23	Java Random-Klasse	43

1 Einführung

Diese Referenz bietet eine Übersicht über die reservierten Schlüsselwörter in Java sowie die wichtigsten Methoden der Klassen `String`, `Math`, `Arrays`, `ArrayList`, `LinkedList`, `HashMap`, `Scanner`, `File`, `FileInputStream`, `FileOutputStream`, `BufferedReader`, `BufferedWriter`, `Iterator`, `Collections`, `System` und häufige Fehler und Ausnahmen. Die Tabellen listen die Schlüsselwörter und Methoden mit ihren Beschreibungen auf. Beispielcode demonstriert die Anwendung ausgewählter Java-Elemente.

2 Java-Schlüsselwörter

Schlüsselwort	Beschreibung
<code>abstract</code>	Nicht-Zugriffsmodifikator. Wird für Klassen und Methoden verwendet: Eine abstrakte Klasse kann keine Objekte erzeugen (muss von einer anderen Klasse geerbt werden). Eine abstrakte Methode kann nur in einer abstrakten Klasse verwendet werden und hat keinen Rumpf. Der Rumpf wird von der Unterklasse bereitgestellt.
<code>assert</code>	Für Debugging-Zwecke.
<code>boolean</code>	Datentyp, der nur <code>true</code> oder <code>false</code> speichern kann.
<code>break</code>	Bricht aus einer Schleife oder einem Switch-Block aus.
<code>byte</code>	Datentyp, der Ganzzahlen von -128 bis 127 speichern kann.
<code>case</code>	Markiert einen Codeblock in einem Switch-Statement.
<code>catch</code>	Fängt Ausnahmen ab, die von Try-Statements generiert werden.
<code>char</code>	Datentyp, der ein einzelnes Zeichen speichert.
<code>class</code>	Definiert eine Klasse.
<code>continue</code>	Führt mit der nächsten Iteration einer Schleife fort.
<code>const</code>	Definiert eine Konstante. Nicht in Gebrauch - stattdessen <code>final</code> verwenden.
<code>default</code>	Gibt den Standard-Codeblock in einem Switch-Statement an.
<code>do</code>	Wird zusammen mit <code>while</code> verwendet, um eine Do-While-Schleife zu erstellen.
<code>double</code>	Datentyp, der Gleitkommazahlen von $1.7 \cdot 10^{-308}$ bis $1.7 \cdot 10^{308}$ speichern kann.
<code>else</code>	Wird in bedingten Anweisungen verwendet.
<code>enum</code>	Deklariert einen unveränderlichen (aufzählenden) Typ.
<code>exports</code>	Exportiert ein Paket mit einem Modul. Neu in Java 9.
<code>extends</code>	Erweitert eine Klasse (zeigt an, dass eine Klasse von einer anderen geerbt wird).
<code>final</code>	Nicht-Zugriffsmodifikator für Klassen, Attribute und Methoden, der sie unveränderbar macht (Vererbung oder Überschreibung unmöglich).
<code>finally</code>	Wird mit Ausnahmen verwendet, ein Codeblock, der immer ausgeführt wird, unabhängig von einer Ausnahme.
<code>float</code>	Datentyp, der Gleitkommazahlen von $3.4 \cdot 10^{-38}$ bis $3.4 \cdot 10^{38}$ speichert.
<code>for</code>	Erstellt eine For-Schleife.
<code>goto</code>	Nicht in Gebrauch, hat keine Funktion.
<code>if</code>	Erstellt eine bedingte Anweisung.
<code>implements</code>	Implementiert ein Interface.
<code>import</code>	Importiert ein Paket, eine Klasse oder ein Interface.
<code>instanceof</code>	Prüft, ob ein Objekt eine Instanz einer bestimmten Klasse oder eines Interfaces ist.
<code>int</code>	Datentyp, der Ganzzahlen von -2147483648 bis 2147483647 speichert.
<code>interface</code>	Deklariert einen speziellen Klassentyp, der nur abstrakte Methoden enthält.
<code>long</code>	Datentyp, der Ganzzahlen von -9223372036854775808 bis 9223372036854775808 speichert.
<code>module</code>	Deklariert ein Modul. Neu in Java 9.
<code>native</code>	Gibt an, dass eine Methode nicht in derselben Java-Quelldatei implementiert ist (sondern in einer anderen Sprache).
<code>new</code>	Erstellt neue Objekte.
<code>package</code>	Deklariert ein Paket.
<code>private</code>	Zugriffsmodifikator für Attribute, Methoden und Konstruktoren, die nur innerhalb der deklarierten Klasse zugänglich sind.
<code>protected</code>	Zugriffsmodifikator für Attribute, Methoden und Konstruktoren, die im selben Paket und in Unterklassen zugänglich sind.
<code>public</code>	Zugriffsmodifikator für Klassen, Attribute, Methoden und Konstruktoren, die von jeder anderen Klasse zugänglich sind.
<code>requires</code>	Gibt erforderliche Bibliotheken in einem Modul an. Neu in Java 9.
<code>return</code>	Beendet die Ausführung einer Methode und kann einen Wert zurückgeben.

Schlüsselwort	Beschreibung
<code>short</code>	Datentyp, der Ganzzahlen von -32768 bis 32767 speichert.
<code>static</code>	Nicht-Zugriffsmodifikator für Methoden und Attribute. Statische Methoden/Attribute können ohne Erstellung eines Klassenobjekts aufgerufen werden.
<code>strictfp</code>	Veraltet. Beschränkt die Präzision und Rundung von Gleitkommaberechnungen.
<code>super</code>	Verweist auf Objekte der Superklasse (Elternklasse).
<code>switch</code>	Wählt einen von vielen Codeblöcken zur Ausführung aus.
<code>synchronized</code>	Nicht-Zugriffsmodifikator, der angibt, dass Methoden nur von einem Thread gleichzeitig aufgerufen werden können.
<code>this</code>	Verweist auf das aktuelle Objekt in einer Methode oder einem Konstruktor.
<code>throw</code>	Erstellt einen benutzerdefinierten Fehler.
<code>throws</code>	Gibt an, welche Ausnahmen eine Methode werfen kann.
<code>transient</code>	Wird verwendet, um ein Attribut bei der Serialisierung eines Objekts zu ignorieren.
<code>try</code>	Erstellt eine Try-Catch-Anweisung.
<code>var</code>	Deklariert eine Variable. Neu in Java 10.
<code>void</code>	Gibt an, dass eine Methode keinen Rückgabewert hat.
<code>volatile</code>	Gibt an, dass ein Attribut nicht lokal im Thread gecacht wird und immer aus dem „Hauptspeicher“ gelesen wird.
<code>while</code>	Erstellt eine While-Schleife.

3 Java String-Methoden

Methode	Rückgabebetyp	Beschreibung	Beispiel
<code>charAt(int index)</code>	<code>char</code>	Gibt das Zeichen an der angegebenen Position (Index) zurück.	<pre> 1 String s = "Java"; 2 char c = s.charAt(1); // 'a' </pre>
<code>codePointAt(int index)</code>	<code>int</code>	Gibt den Unicode-Wert des Zeichens an der angegebenen Position zurück.	<pre> 1 String s = "A"; 2 int cp = s.codePointAt(0); // 65 </pre>
<code>codePointBefore(int index)</code>	<code>int</code>	Gibt den Unicode-Wert des Zeichens vor der angegebenen Position zurück.	<pre> 1 String s = "Java"; 2 int cp = s.codePointBefore(2); // 'v' -> 118 </pre>
<code>codePointCount(int beginIndex, int endIndex)</code>	<code>int</code>	Gibt die Anzahl der Unicode-Werte in einem String zurück.	<pre> 1 String s = "a\uD83D\ uDE00b"; 2 int cnt = s.codePointCount(0, s.length()); // 3 </pre>
<code>compareTo(String anotherString)</code>	<code>int</code>	Vergleicht zwei Strings lexikografisch.	<pre> 1 int cmp = "apple".compareTo("banana"); // negativ </pre>
<code>compareToIgnoreCase(String str)</code>	<code>int</code>	Vergleicht zwei Strings lexikografisch, ohne Groß-/Kleinschreibung zu berücksichtigen.	<pre> 1 int cmp = "Apple".compareToIgnoreCase("apple"); // 0 </pre>
<code>concat(String str)</code>	<code>String</code>	Fügt einen String an das Ende eines anderen Strings an.	<pre> 1 String s = "Hello".concat(" World"); // "Hello World" </pre>
<code>contains(CharSequence s)</code>	<code>boolean</code>	Prüft, ob ein String eine Zeichenfolge enthält.	<pre> 1 boolean b = "Java".contains("av"); // true </pre>
<code>contentEquals(CharSequence cs)</code>	<code>boolean</code>	Prüft, ob ein String exakt dieselbe Zeichenfolge wie eine angegebene <code>CharSequence</code> oder <code>StringBuffer</code> enthält.	<pre> 1 boolean eq = "abc".contentEquals(new StringBuffer("abc")); // true </pre>
<code>copyValueOf(char[] data)</code>	<code>String</code>	Gibt einen String zurück, der die Zeichen eines Zeichenarrays darstellt.	<pre> 1 char[] arr = {'a','b','c'}; 2 String s = String.copyValueOf(arr); // "abc" </pre>

Fortsetzung auf nächster Seite...

Methoden	Rückgabotyp	Beschreibung	Beispiel
<code>endsWith(String suffix)</code>	<code>boolean</code>	Prüft, ob ein String mit den angegebenen Zeichen endet.	<pre>boolean b = "file.txt". endsWith(".txt"); // true</pre>
<code>equals(Object anObject)</code>	<code>boolean</code>	Vergleicht zwei Strings. Gibt <code>true</code> zurück, wenn die Strings gleich sind, sonst <code>false</code> .	<pre>boolean eq = "test". equals("test"); // true</pre>
<code>equalsIgnoreCase(String anotherString)</code>	<code>boolean</code>	Vergleicht zwei Strings, ohne Groß-/Kleinschreibung zu berücksichtigen.	<pre>boolean eq = "Test". equalsIgnoreCase(" test"); // true</pre>
<code>format(String format, Object... args)</code>	<code>String</code>	Gibt einen formatierten String mit der angegebenen Lokalisierung, Formatzeichenkette und Argumenten zurück.	<pre>String s = String.format ("Pi: %.2f", Math. PI); // "Pi: 3,14"</pre>
<code>getBytes()</code>	<code>byte[]</code>	Konvertiert einen String in ein Byte-Array.	<pre>byte[] b = "abc". getBytes(); // [97,98,99]</pre>
<code>getChars(int srcBegin, int srcEnd, char[] dst, int dstBegin)</code>	<code>void</code>	Kopiert Zeichen aus einem String in ein Zeichen-Array.	<pre>char[] dst = new char [3]; "Java".getChars(0, 3, dst, 0); // dst = ['J','a','v']</pre>
<code>hashCode()</code>	<code>int</code>	Gibt den Hash-Code eines Strings zurück.	<pre>int h = "hello".hashCode ();</pre>
<code>indexOf(int ch)</code>	<code>int</code>	Gibt die Position des ersten Auftretens angegebener Zeichen in einem String zurück.	<pre>int i = "Java".indexOf(' a'); // 1</pre>
<code>intern()</code>	<code>String</code>	Gibt die kanonische Darstellung des String-Objekts zurück.	<pre>String s = new String(" test").intern();</pre>
<code>isEmpty()</code>	<code>boolean</code>	Prüft, ob ein String leer ist.	<pre>boolean empty = "". isEmpty(); // true</pre>
<code>join(CharSequence delimiter, CharSequence... elements)</code>	<code>String</code>	Verbindet einen oder mehrere Strings mit einem angegebenen Trennzeichen.	<pre>String s = String.join(" , ", "a", "b", "c") ; // "a, b, c"</pre>
<code>lastIndexOf(int ch)</code>	<code>int</code>	Gibt die Position des letzten Auftretens angegebener Zeichen in einem String zurück.	<pre>int i = "banana". lastIndexOf('a'); // 5</pre>

Fortsetzung auf nächster Seite...

Methoden	Rückgabotyp	Beschreibung	Beispiel
<code>length()</code>	<code>int</code>	Gibt die Länge eines Strings zurück.	<pre>int len = "Hello".length(); // 5</pre>
<code>matches(String regex)</code>	<code>boolean</code>	Sucht in einem String nach einem Treffer mit einem regulären Ausdruck und gibt die Treffer zurück.	<pre>boolean m = "123".matches("\\d+"); // true</pre>
<code>offsetByCodePoints(int index, int codePointOffset)</code>	<code>int</code>	Gibt den Index innerhalb dieses Strings zurück, der um <code>codePointOffset</code> Codepunkte vom angegebenen Index verschoben ist.	<pre>int i = "a\uD83D\uDE00b".offsetByCodePoints(0, 1); // 1</pre>
<code>regionMatches(int toffset, String other, int ooffset, int len)</code>	<code>boolean</code>	Prüft, ob zwei String-Bereiche gleich sind.	<pre>boolean eq = "Java".regionMatches(1, "ava", 0, 3); // true</pre>
<code>replace(char oldChar, char newChar)</code>	<code>String</code>	Sucht in einem String nach einem angegebenen Wert und gibt einen neuen String zurück, bei dem die angegebenen Werte ersetzt wurden.	<pre>String s = "foo".replace('o', 'a'); // "faa"</pre>
<code>replaceAll(String regex, String replacement)</code>	<code>String</code>	Ersetzt jede Teilzeichenfolge dieses Strings, die mit dem angegebenen regulären Ausdruck übereinstimmt, durch den angegebenen Ersatz.	<pre>String s = "a b c".replaceAll("\\s+", ","); // "a,b,c"</pre>
<code>replaceFirst(String regex, String replacement)</code>	<code>String</code>	Ersetzt das erste Auftreten einer Teilzeichenfolge, die mit dem angegebenen regulären Ausdruck übereinstimmt, durch den angegebenen Ersatz.	<pre>String s = "abc abc".replaceFirst("abc", "xyz"); // "xyzabc"</pre>
<code>split(String regex)</code>	<code>String[]</code>	Teilt einen String in ein Array von Teilstrings.	<pre>String[] arr = "a,b,c".split(","); // ["a", "b", "c"]</pre>
<code>startsWith(String prefix)</code>	<code>boolean</code>	Prüft, ob ein String mit den angegebenen Zeichen beginnt.	<pre>boolean b = "hello.txt".startsWith("hello"); // true</pre>
<code>subSequence(int beginIndex, int endIndex)</code>	<code>CharSequence</code>	Gibt eine neue Zeichenfolge zurück, die eine Teilsequenz dieser Sequenz ist.	<pre>CharSequence cs = "Java".subSequence(1, 3); // "av"</pre>
<code>substring(int beginIndex)</code>	<code>String</code>	Gibt einen neuen String zurück, der eine Teilzeichenfolge eines angegebenen Strings ist.	<pre>String sub = "HelloWorld".substring(6); // "World"</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
toCharArray()	char[]	Konvertiert diesen String in ein neues Zeichen-Array.	<pre>char[] arr = "abc".toCharArray(); // ['a', 'b', 'c']</pre>
toLowerCase()	String	Konvertiert einen String in Kleinbuchstaben.	<pre>String s = "ABC".toLowerCase(); // "abc"</pre>
toString()	String	Gibt den Wert eines String-Objekts zurück.	<pre>String s = "test".toString(); // "test"</pre>
toUpperCase()	String	Konvertiert einen String in Großbuchstaben.	<pre>String s = "abc".toUpperCase(); // "ABC"</pre>
trim()	String	Entfernt Leerzeichen von beiden Enden eines Strings.	<pre>String s = " abc ".trim(); // "abc"</pre>
valueOf(Object obj)	String	Gibt die String-Darstellung des angegebenen Werts zurück.	<pre>String s = String.valueOf(123); // "123"</pre>

4 Java Math-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>abs(double a)</code>	<code>double</code>	Gibt den absoluten Wert von <code>a</code> zurück.	<pre>double abs = Math.abs(-5.7); // 5.7</pre>
<code>acos(double a)</code>	<code>double</code>	Gibt den Arkuskosinus von <code>a</code> in Radiant zurück.	<pre>double ac = Math.acos(0.5); // PI/3</pre>
<code>addExact(int x, int y)</code>	<code>int</code>	Gibt die Summe von <code>x</code> und <code>y</code> zurück.	<pre>int sum = Math.addExact(10, 20); // 30</pre>
<code>asin(double a)</code>	<code>double</code>	Gibt den Arkussinus von <code>a</code> in Radiant zurück.	<pre>double as = Math.asin(0.5); // PI/6</pre>
<code>atan(double a)</code>	<code>double</code>	Gibt den Arkustangens von <code>a</code> als numerischer Wert zwischen $-\pi/2$ und $\pi/2$ Radiant zurück.	<pre>double at = Math.atan(1.0); // PI/4</pre>
<code>atan2(double y, double x)</code>	<code>double</code>	Gibt den Winkel <code>theta</code> aus der Umwandlung von kartesischen Koordinaten (<code>x</code> , <code>y</code>) in Polarkoordinaten (<code>r</code> , <code>theta</code>) zurück.	<pre>double theta = Math.atan2(1, 1); // PI/4</pre>
<code>cbrt(double a)</code>	<code>double</code>	Gibt die Kubikwurzel von <code>a</code> zurück.	<pre>double cb = Math.cbrt(27.0); // 3.0</pre>
<code>ceil(double a)</code>	<code>double</code>	Gibt den Wert von <code>a</code> auf die nächsthöhere Ganzzahl gerundet zurück.	<pre>double c = Math.ceil(4.3); // 5.0</pre>
<code>copySign(double magnitude, double sign)</code>	<code>double</code>	Gibt den ersten Gleitkommawert <code>magnitude</code> mit dem Vorzeichen des zweiten Gleitkommawerts <code>sign</code> zurück.	<pre>double cs = Math.copySign(3.0, -1.0); // -3.0</pre>
<code>cos(double a)</code>	<code>double</code>	Gibt den Kosinus von <code>a</code> (<code>a</code> in Radiant) zurück.	<pre>double co = Math.cos(0); // 1.0</pre>
<code>cosh(double x)</code>	<code>double</code>	Gibt den hyperbolischen Kosinus eines <code>double</code> -Werts zurück.	<pre>double ch = Math.cosh(0); // 1.0</pre>
<code>decrementExact(int a)</code>	<code>int</code>	Gibt <code>a-1</code> zurück.	<pre>int dec = Math.decrementExact(5); // 4</pre>
<code>exp(double a)</code>	<code>double</code>	Gibt den Wert von e^a zurück.	<pre>double e = Math.exp(1); // ca. 2.718</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
expm1(double x)	double	Gibt $e^x - 1$ zurück.	<pre>double em = Math.expm1(0); // 0.0</pre>
floor(double a)	double	Gibt den Wert von a auf die nächstniedrigere Ganzzahl gerundet zurück.	<pre>double f = Math.floor(4.7); // 4.0</pre>
floorDiv(int x, int y)	int	Gibt die Division zwischen x und y abgerundet zurück.	<pre>int fd = Math.floorDiv(7, 3); // 2</pre>
floorMod(int x, int y)	int	Gibt den Rest einer Division zwischen x und y zurück, wobei das Ergebnis der Division abgerundet wurde.	<pre>int fm = Math.floorMod(7, 3); // 1</pre>
getExponent(double d)	int	Gibt den unverzerrten Exponenten von d zurück.	<pre>int exp = Math.getExponent(8.0); // 3</pre>
hypot(double x, double y)	double	Gibt $\sqrt{x^2 + y^2}$ ohne Zwischenüberlauf oder -unterlauf zurück.	<pre>double h = Math.hypot(3, 4); // 5.0</pre>
IEEERemainder(double f1, double f2)	double	Berechnet die Restoperation von f1 und f2 gemäß dem IEEE 754 Standard.	<pre>double r = Math.IEERemainder(10, 3); // 1.0</pre>
incrementExact(int a)	int	Gibt a+1 zurück.	<pre>int inc = Math.incrementExact(5); // 6</pre>
log(double a)	double	Gibt den natürlichen Logarithmus (Basis e) von a zurück.	<pre>double ln = Math.log(Math.E); // 1.0</pre>
log10(double a)	double	Gibt den Logarithmus zur Basis 10 von a zurück.	<pre>double l10 = Math.log10(100); // 2.0</pre>
log1p(double x)	double	Gibt den natürlichen Logarithmus der Summe von x und 1 zurück.	<pre>double lp = Math.log1p(0); // 0.0</pre>
max(double a, double b)	double	Gibt den höchsten Wert von a und b zurück.	<pre>double m = Math.max(10, 20); // 20.0</pre>
min(double a, double b)	double	Gibt den niedrigsten Wert von a und b zurück.	<pre>double m = Math.min(10, 20); // 10.0</pre>
multiplyExact(int x, int y)	int	Gibt das Ergebnis von x multipliziert mit y zurück.	<pre>int prod = Math.multiplyExact(3, 4); // 12</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>negateExact(int a)</code>	<code>int</code>	Gibt die Negation von <code>a</code> zurück.	<pre>int neg = Math. negateExact(5); // -5</pre>
<code>nextAfter(double start, double direction)</code>	<code>double</code>	Gibt die Gleitkommazahl zurück, die an <code>start</code> in Richtung <code>direction</code> angrenzt.	<pre>double na = Math. nextAfter(1.0, 2.0) ; // naechster Wert >1.0</pre>
<code>nextDown(double d)</code>	<code>double</code>	Gibt den Gleitkommawert zurück, der an <code>d</code> in negativer Richtung angrenzt.	<pre>double nd = Math. nextDown(1.0); // naechster Wert <1.0</pre>
<code>nextUp(double d)</code>	<code>double</code>	Gibt den Gleitkommawert zurück, der an <code>d</code> in Richtung positiver Unendlichkeit angrenzt.	<pre>double nu = Math.nextUp (1.0); // naechster Wert >1.0</pre>
<code>pow(double a, double b)</code>	<code>double</code>	Gibt den Wert von <code>a</code> hoch <code>b</code> zurück.	<pre>double p = Math.pow(2, 3); // 8.0</pre>
<code>random()</code>	<code>double</code>	Gibt eine Zufallszahl zwischen 0 und 1 zurück.	<pre>double r = Math.random() ; // 0.0 <= r < 1.0</pre>
<code>rint(double a)</code>	<code>double</code>	Gibt den <code>double</code> -Wert zurück, der <code>a</code> am nächsten ist und einer mathematischen Ganzzahl entspricht.	<pre>double ri = Math.rint (4.5); // 4.0 oder 5.0</pre>
<code>round(double a)</code>	<code>long</code>	Gibt den Wert von <code>a</code> auf die nächstliegende Ganzzahl gerundet zurück.	<pre>long rnd = Math.round (4.6); // 5</pre>
<code>scalb(double d, int scaleFactor)</code>	<code>double</code>	Gibt <code>d</code> multipliziert mit 2 hoch <code>scaleFactor</code> zurück.	<pre>double s = Math.scalb (1.0, 3); // 8.0</pre>
<code>signum(double d)</code>	<code>double</code>	Gibt das Vorzeichen von <code>d</code> zurück.	<pre>double sg = Math.signum (-5.0); // -1.0</pre>
<code>sin(double a)</code>	<code>double</code>	Gibt den Sinus von <code>a</code> (<code>a</code> in Radiant) zurück.	<pre>double si = Math.sin(Math.PI/2); // 1.0</pre>
<code>sinh(double x)</code>	<code>double</code>	Gibt den hyperbolischen Sinus eines <code>double</code> -Werts zurück.	<pre>double sh = Math.sinh(0) ; // 0.0</pre>
<code>sqrt(double a)</code>	<code>double</code>	Gibt die Quadratwurzel von <code>a</code> zurück.	<pre>double sq = Math.sqrt (16); // 4.0</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>subtractExact(int x, int y)</code>	<code>int</code>	Gibt das Ergebnis von x minus y zurück.	<pre>int sub = Math. subtractExact(20, 10); // 10</pre>
<code>tan(double a)</code>	<code>double</code>	Gibt den Tangens eines Winkels zurück.	<pre>double t = Math.tan(Math .PI/4); // 1.0</pre>
<code>tanh(double x)</code>	<code>double</code>	Gibt den hyperbolischen Tangens eines <code>double</code> -Werts zurück.	<pre>double th = Math.tanh(0) ; // 0.0</pre>
<code>toDegrees(double anggrad)</code>	<code>double</code>	Konvertiert einen Winkel in Radiant in einen ungefähren Winkel in Grad.	<pre>double deg = Math. toDegrees(Math.PI); // 180.0</pre>
<code>toIntExact(long value)</code>	<code>int</code>	Konvertiert einen <code>long</code> -Wert in einen <code>int</code> -Wert.	<pre>int i = Math.toIntExact (100L); // 100</pre>
<code>toRadians(double angdeg)</code>	<code>double</code>	Konvertiert einen Winkel in Grad in einen ungefähren Winkel in Radiant.	<pre>double rad = Math. toRadians(180); // PI</pre>
<code>ulp(double d)</code>	<code>double</code>	Gibt die Größe der Einheit der geringsten Präzision (ulp) von d zurück.	<pre>double u = Math.ulp(1.0) ;</pre>

5 Java Ausgabe-Methoden

Methode	Rückgabebetyp	Beschreibung	Beispiel
<code>print(Object obj)</code>	<code>void</code>	Gibt Text oder Werte auf der Konsole aus.	<pre>System.out.print("Hallo"); // Hallo</pre>
<code>printf(String format, Object... args)</code>	<code>PrintStream</code>	Gibt formatierten Text oder Werte auf der Konsole aus.	<pre>System.out.printf("Zahl: %d\n", 42); // Zahl: 42</pre>
<code>println(Object obj)</code>	<code>void</code>	Gibt Text oder Werte auf der Konsole aus, gefolgt von einer neuen Zeile.	<pre>System.out.println("Welt"); // Welt\n</pre>

6 Java Arrays-Methoden

Methode	Rückgabebetyp	Beschreibung	Beispiel
<code>compare(int[] a, int[] b)</code>	<code>int</code>	Vergleicht zwei Arrays.	<pre> 1 int[] a = {1,2}; int[] b = {1,3}; 2 int cmp = Arrays.compare (a, b); // negativ </pre>
<code>copyOf(int[] original, int newLength)</code>	<code>int[]</code>	Erstellt eine Kopie eines Arrays mit einer neuen Länge.	<pre> 1 int[] orig = {1,2,3}; 2 int[] copy = Arrays. copyOf(orig, 5); // {1,2,3,0,0} </pre>
<code>deepEquals(Object[] a1, Object[] a2)</code>	<code>boolean</code>	Vergleicht zwei mehrdimensionale Arrays, um zu prüfen, ob sie tief gleich sind.	<pre> 1 int[][] a = {{1}}, b = {{1}}; 2 boolean eq = Arrays. deepEquals(a, b); // true </pre>
<code>equals(int[] a, int[] b)</code>	<code>boolean</code>	Prüft, ob zwei Arrays gleich sind.	<pre> 1 int[] a = {1,2}, b = {1,2}; 2 boolean eq = Arrays. equals(a, b); // true </pre>
<code>fill(int[] a, int val)</code>	<code>void</code>	Füllt ein Array mit einem angegebenen Wert.	<pre> 1 int[] arr = new int[3]; 2 Arrays.fill(arr, 9); // {9,9,9} </pre>
<code>mismatch(int[] a, int[] b)</code>	<code>int</code>	Gibt die Indexposition des ersten Konflikts zwischen zwei Arrays zurück.	<pre> 1 int[] a = {1,2,3}, b = {1,4,3}; 2 int m = Arrays.mismatch(a, b); // 1 </pre>
<code>sort(int[] a)</code>	<code>void</code>	Sortiert ein Array in aufsteigender Reihenfolge.	<pre> 1 int[] arr = {5,2,8}; 2 Arrays.sort(arr); // {2,5,8} </pre>

7 Java ArrayList-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>add(E e)</code>	<code>boolean</code>	Fügt ein Element zur Liste hinzu.	<pre> 1 ArrayList<String> list = new ArrayList<>(); 2 list.add("Apfel"); // true </pre>
<code>addAll(Collection<? extends E> c)</code>	<code>boolean</code>	Fügt eine Sammlung von Elementen zur Liste hinzu.	<pre> 1 list.addAll(Arrays. asList("Banane", " Kirsche")); // true </pre>
<code>clear()</code>	<code>void</code>	Entfernt alle Elemente aus der Liste.	<pre> 1 list.clear(); // list leer </pre>
<code>clone()</code>	<code>Object</code>	Erstellt eine Kopie der ArrayList.	<pre> 1 ArrayList<String> copy = (ArrayList<String> >) list.clone(); </pre>
<code>contains(Object o)</code>	<code>boolean</code>	Prüft, ob ein Element in der Liste vorhanden ist.	<pre> 1 boolean has = list. contains("Apfel"); // true </pre>
<code>ensureCapacity(int minCapacity)</code>	<code>void</code>	Erhöht die Kapazität der Liste, um eine angegebene Anzahl von Elementen aufnehmen zu können.	<pre> 1 list.ensureCapacity(100) ; </pre>
<code>forEach(Consumer<? super E> action)</code>	<code>void</code>	Führt eine Aktion für jedes Element in der Liste aus.	<pre> 1 list.forEach(System.out ::println); </pre>
<code>get(int index)</code>	<code>E</code>	Gibt das Element an einer bestimmten Position in der Liste zurück.	<pre> 1 String s = list.get(0); // "Apfel" </pre>
<code>indexOf(Object o)</code>	<code>int</code>	Gibt die Position des ersten Auftretens eines Elements in der Liste zurück.	<pre> 1 int idx = list.indexOf(" Banane"); // 1 </pre>
<code>isEmpty()</code>	<code>boolean</code>	Prüft, ob die Liste leer ist.	<pre> 1 boolean empty = list. isEmpty(); // false </pre>
<code>iterator()</code>	<code>Iterator<E></code>	Gibt ein Iterator-Objekt für die ArrayList zurück.	<pre> 1 Iterator<String> it = list.iterator(); </pre>
<code>lastIndexOf(Object o)</code>	<code>int</code>	Gibt die Position des letzten Auftretens eines Elements in der Liste zurück.	<pre> 1 list.add("Apfel"); 2 int last = list. lastIndexOf("Apfel"); // 2 </pre>

Fortsetzung auf nächster Seite...

Methoden	Rückgabebetyp	Beschreibung	Beispiel
<code>listIterator()</code>	<code>ListIterator<E></code>	Gibt ein <code>ListIterator</code> -Objekt für die <code>ArrayList</code> zurück.	<pre>ListIterator<String> lit = list. listIterator();</pre>
<code>remove(int index)</code>	<code>E</code>	Entfernt ein Element aus der Liste.	<pre>String rem = list.remove (0); // "Apfel"</pre>
<code>removeAll(Collection<?> c)</code>	<code>boolean</code>	Entfernt eine Sammlung von Elementen aus der Liste.	<pre>list.removeAll(Arrays. asList("Banane")); // true</pre>
<code>removeIf(Predicate<? super E> filter)</code>	<code>boolean</code>	Entfernt alle Elemente aus der Liste, die eine angegebene Bedingung erfüllen.	<pre>list.removeIf(s -> s. startsWith("A")); // true</pre>
<code>replaceAll(UnaryOperator<E> operator)</code>	<code>void</code>	Ersetzt jedes Element in der Liste durch das Ergebnis einer Operation auf diesem Element.	<pre>list.replaceAll(String:: toUpperCase);</pre>
<code>retainAll(Collection<?> c)</code>	<code>boolean</code>	Entfernt alle Elemente aus der Liste, die nicht zu einer angegebenen Sammlung gehören.	<pre>list.retainAll(Arrays. asList("Apfel")); // true</pre>
<code>set(int index, E element)</code>	<code>E</code>	Ersetzt ein Element an einer bestimmten Position in der Liste.	<pre>String old = list.set(0, "Orange"); // alter Wert</pre>
<code>size()</code>	<code>int</code>	Gibt die Anzahl der Elemente in der Liste zurück.	<pre>int sz = list.size(); // z. B. 3</pre>
<code>sort(Comparator<? super E> c)</code>	<code>void</code>	Sortiert die Liste.	<pre>list.sort(String:: compareTo);</pre>
<code>splitIterator()</code>	<code>SplitIterator<E></code>	Gibt ein <code>SplitIterator</code> -Objekt für die <code>ArrayList</code> zurück.	<pre>SplitIterator<String> sp = list.splitIterator() ;</pre>
<code>subList(int fromIndex, int toIndex)</code>	<code>List<E></code>	Gibt eine Teilliste zurück, die Zugriff auf einen Bereich der Elemente dieser Liste bietet.	<pre>List<String> sub = list. subList(0, 2);</pre>
<code>toArray()</code>	<code>Object[]</code>	Gibt ein Array zurück, das die Elemente der Liste enthält.	<pre>Object[] arr = list. toArray();</pre>
<code>trimToSize()</code>	<code>void</code>	Reduziert die Kapazität der Liste auf die Anzahl der Elemente, falls erforderlich.	<pre>list.trimToSize();</pre>

8 Java LinkedList-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>add(E e)</code>	<code>boolean</code>	Fügt ein Element zur Liste hinzu.	<pre> 1 LinkedList<String> ll = 2 new LinkedList<>(); 3 ll.add("A"); // true </pre>
<code>addAll(Collection<? extends E> c)</code>	<code>boolean</code>	Fügt eine Sammlung von Elementen zur Liste hinzu.	<pre> 1 ll.addAll(Arrays.asList(2 "B", "C")); // true </pre>
<code>addFirst(E e)</code>	<code>void</code>	Fügt ein Element an den Anfang der Liste hinzu.	<pre> 1 ll.addFirst("X"); </pre>
<code>addLast(E e)</code>	<code>void</code>	Fügt ein Element ans Ende der Liste hinzu.	<pre> 1 ll.addLast("Z"); </pre>
<code>clear()</code>	<code>void</code>	Entfernt alle Elemente aus der Liste.	<pre> 1 ll.clear(); </pre>
<code>clone()</code>	<code>Object</code>	Erstellt eine Kopie der LinkedList.	<pre> 1 LinkedList<String> copy 2 = (LinkedList< 3 String>) ll.clone() 4 ; </pre>
<code>contains(Object o)</code>	<code>boolean</code>	Prüft, ob ein Element in der Liste vorhanden ist.	<pre> 1 boolean has = ll. 2 contains("A"); </pre>
<code>descendingIterator()</code>	<code>Iterator<E></code>	Gibt einen Iterator zurück, um die Elemente der Liste in umgekehrter Reihenfolge zu durchlaufen.	<pre> 1 Iterator<String> it = ll 2 .descendingIterator() 3 ; </pre>
<code>element()</code>	<code>E</code>	Ruft das erste Element in der Liste ab. Ähnlich wie <code>getFirst()</code> .	<pre> 1 String first = ll. 2 element(); </pre>
<code>forEach(Consumer<? super E> action)</code>	<code>void</code>	Führt eine Aktion für jedes Element in der Liste aus.	<pre> 1 ll.forEach(System.out:: 2 println); </pre>
<code>get(int index)</code>	<code>E</code>	Gibt das Element an einer bestimmten Position in der Liste zurück.	<pre> 1 String s = ll.get(1); </pre>
<code>getFirst()</code>	<code>E</code>	Gibt das erste Element in der Liste zurück.	<pre> 1 String f = ll.getFirst() 2 ; </pre>
<code>getLast()</code>	<code>E</code>	Gibt das letzte Element in der Liste zurück.	<pre> 1 String l = ll.getLast(); </pre>
<code>indexOf(Object o)</code>	<code>int</code>	Gibt die Position des ersten Auftretens eines Elements in der Liste zurück.	<pre> 1 int idx = ll.indexOf("B") 2 ; </pre>

Fortsetzung auf nächster Seite...

Methoden	Rückgabotyp	Beschreibung	Beispiel
<code>isEmpty()</code>	<code>boolean</code>	Prüft, ob die Liste leer ist.	<pre>boolean empty = ll. isEmpty();</pre>
<code>iterator()</code>	<code>Iterator<E></code>	Gibt ein <code>Iterator</code> -Objekt für die <code>LinkedList</code> zurück.	<pre>Iterator<String> it = ll. .iterator();</pre>
<code>lastIndexOf(Object o)</code>	<code>int</code>	Gibt die Position des letzten Auftretens eines Elements in der Liste zurück.	<pre>int last = ll. lastIndexOf("A");</pre>
<code>listIterator()</code>	<code>ListIterator<E></code>	Gibt ein <code>ListIterator</code> -Objekt für die <code>LinkedList</code> zurück.	<pre>ListIterator<String> lit = ll.listIterator ();</pre>
<code>offer(E e)</code>	<code>boolean</code>	Fügt ein Element ans Ende der Liste hinzu.	<pre>ll.offer("Y"); // true</pre>
<code>offerFirst(E e)</code>	<code>boolean</code>	Fügt ein Element an den Anfang der Liste hinzu.	<pre>ll.offerFirst("Start"); // true</pre>
<code>offerLast(E e)</code>	<code>boolean</code>	Fügt ein Element ans Ende der Liste hinzu.	<pre>ll.offerLast("End"); // true</pre>
<code>peek()</code>	<code>E</code>	Ruft das erste Element in der Liste ab. Ähnlich wie <code>getFirst()</code> .	<pre>String p = ll.peek();</pre>
<code>peekFirst()</code>	<code>E</code>	Ruft das erste Element in der Liste ab. Ähnlich wie <code>peek()</code> .	<pre>String pf = ll.peekFirst ();</pre>
<code>peekLast()</code>	<code>E</code>	Ruft das letzte Element in der Liste ab.	<pre>String pl = ll.peekLast ();</pre>
<code>poll()</code>	<code>E</code>	Ruft das erste Element in der Liste ab und entfernt es.	<pre>String rem = ll.poll();</pre>
<code>pollFirst()</code>	<code>E</code>	Ruft das erste Element in der Liste ab und entfernt es. Ähnlich wie <code>poll()</code> .	<pre>String rf = ll.pollFirst ();</pre>
<code>pollLast()</code>	<code>E</code>	Ruft das letzte Element in der Liste ab und entfernt es.	<pre>String rl = ll.pollLast ();</pre>
<code>pop()</code>	<code>E</code>	Gibt das erste Element in der Liste zurück. Ähnlich wie <code>removeFirst()</code> .	<pre>String pop = ll.pop();</pre>
<code>push(E e)</code>	<code>void</code>	Fügt ein Element an den Anfang der Liste hinzu. Ähnlich wie <code>addFirst()</code> .	<pre>ll.push("Top");</pre>
<code>remove()</code>	<code>E</code>	Entfernt ein Element aus der Liste.	<pre>String r = ll.remove();</pre>

Fortsetzung auf nächster Seite...

Methoden	Rückgabotyp	Beschreibung	Beispiel
<code>removeAll(Collection<?> c)</code>	<code>boolean</code>	Entfernt eine Sammlung von Elementen aus der Liste.	<pre>ll.removeAll(Arrays.asList("A")); // true</pre>
<code>removeFirst()</code>	<code>E</code>	Entfernt das erste Element aus der Liste.	<pre>String rf = ll.removeFirst();</pre>
<code>removeFirstOccurrence(Object o)</code>	<code>boolean</code>	Entfernt das erste Auftreten eines angegebenen Elements in der Liste.	<pre>boolean rem = ll.removeFirstOccurrence("A");</pre>
<code>removeIf(Predicate<? super E> filter)</code>	<code>boolean</code>	Entfernt alle Elemente aus der Liste, die eine angegebene Bedingung erfüllen.	<pre>ll.removeIf(s -> s.length() > 1);</pre>
<code>removeLast()</code>	<code>E</code>	Entfernt das letzte Element aus der Liste.	<pre>String rl = ll.removeLast();</pre>
<code>removeLastOccurrence(Object o)</code>	<code>boolean</code>	Entfernt das letzte Auftreten eines angegebenen Elements in der Liste.	<pre>boolean rem = ll.removeLastOccurrence("A");</pre>
<code>replaceAll(UnaryOperator<E> operator)</code>	<code>void</code>	Ersetzt jedes Element in der Liste mit dem Ergebnis einer Operation auf diesem Element.	<pre>ll.replaceAll(String::toUpperCase);</pre>
<code>retainAll(Collection<?> c)</code>	<code>boolean</code>	Entfernt alle Elemente aus der Liste, die nicht zu einer angegebenen Sammlung gehören.	<pre>ll.retainAll(Arrays.asList("B"));</pre>
<code>set(int index, E element)</code>	<code>E</code>	Ersetzt ein Element an einer bestimmten Position in der Liste.	<pre>String old = ll.set(0, "Neu");</pre>
<code>size()</code>	<code>int</code>	Gibt die Anzahl der Elemente in der Liste zurück.	<pre>int sz = ll.size();</pre>
<code>sort(Comparator<? super E> c)</code>	<code>void</code>	Sortiert die Liste.	<pre>ll.sort(String::compareTo);</pre>
<code>splitter()</code>	<code>Splitter<E></code>	Gibt ein Splitter-Objekt für die LinkedList zurück.	<pre>Splitter<String> sp = ll.splitter();</pre>
<code>subList(int fromIndex, int toIndex)</code>	<code>List<E></code>	Gibt eine Teilliste zurück, die Zugriff auf einen Bereich der Elemente dieser Liste bietet.	<pre>List<String> sub = ll.subList(0, 2);</pre>
<code>toArray()</code>	<code>Object[]</code>	Gibt ein Array zurück, das die Elemente der Liste enthält.	<pre>Object[] arr = ll.toArray();</pre>

9 Java HashMap-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>clear()</code>	<code>void</code>	Entfernt alle Einträge aus der Map.	<pre> 1 HashMap<String,Integer> map = new HashMap <>(); 2 map.put("A",1); 3 map.clear(); // leer </pre>
<code>clone()</code>	<code>Object</code>	Erstellt eine Kopie der HashMap.	<pre> 1 HashMap<String,Integer> copy = (HashMap< String,Integer>) map.clone(); </pre>
<code>compute(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)</code>	<code>V</code>	Berechnet einen Wert für einen Eintrag basierend auf seinem Schlüssel und dem aktuellen Wert (falls vorhanden).	<pre> 1 map.compute("A", (k,v) -> v == null ? 1 : v+1); </pre>
<code>computeIfAbsent(K key, Function<? super K,? extends V> mappingFunction)</code>	<code>V</code>	Berechnet einen Wert für einen Eintrag basierend auf seinem Schlüssel, nur wenn kein Eintrag mit dem Schlüssel existiert.	<pre> 1 map.computeIfAbsent("B", k -> 2); </pre>
<code>computeIfPresent(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)</code>	<code>V</code>	Berechnet einen neuen Wert für einen Eintrag basierend auf seinem Schlüssel und aktuellen Wert, nur wenn ein Eintrag mit dem Schlüssel existiert.	<pre> 1 map.computeIfPresent("A", (k,v) -> v+1); </pre>
<code>containsKey(Object key)</code>	<code>boolean</code>	Gibt an, ob ein Eintrag mit dem angegebenen Schlüssel in der Map existiert.	<pre> 1 boolean has = map. containsKey("A"); </pre>
<code>containsValue(Object value)</code>	<code>boolean</code>	Gibt an, ob ein Eintrag mit dem angegebenen Wert in der Map existiert.	<pre> 1 boolean has = map. containsValue(1); </pre>
<code>entrySet()</code>	<code>Set<Map.Entry<K,V>></code>	Gibt eine Menge aller Einträge in der Map zurück.	<pre> 1 Set<Map.Entry<String, Integer>> set = map .entrySet(); </pre>
<code>forEach(BiConsumer<? super K,? super V> action)</code>	<code>void</code>	Führt eine Aktion für jeden Eintrag in der Map aus.	<pre> 1 map.forEach((k,v) -> System.out.println(k + "=" + v)); </pre>
<code>get(Object key)</code>	<code>V</code>	Gibt den Wert des Eintrags mit einem angegebenen Schlüssel zurück.	<pre> 1 Integer v = map.get("A") ; </pre>
<code>getOrDefault(Object key, V defaultValue)</code>	<code>V</code>	Gibt den Wert des Eintrags mit einem angegebenen Schlüssel oder einen Standardwert zurück, wenn der Eintrag nicht gefunden wird.	<pre> 1 Integer v = map. getOrDefault("X", 0); </pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>isEmpty()</code>	<code>boolean</code>	Gibt an, ob die Map leer ist.	<pre>1 boolean empty = map. isEmpty();</pre>
<code>keySet()</code>	<code>Set<K></code>	Gibt eine Menge aller Schlüssel in der Map zurück.	<pre>1 Set<String> keys = map. keySet();</pre>
<code>merge(K key, V value, BiFunction<? super V, ? super V, ? extends V> remappingFunction)</code>	<code>V</code>	Berechnet einen Wert für einen Eintrag basierend auf seinem Schlüssel und Wert oder schreibt einen bestimmten Wert, wenn der Eintrag noch nicht existiert.	<pre>1 map.merge("A", 1, Integer::sum);</pre>
<code>put(K key, V value)</code>	<code>V</code>	Schreibt einen Eintrag in die Map.	<pre>1 map.put("A", 10);</pre>
<code>putAll(Map<? extends K, ? extends V> m)</code>	<code>void</code>	Schreibt alle Einträge aus einer anderen Map in diese.	<pre>1 map.putAll(anotherMap);</pre>
<code>putIfAbsent(K key, V value)</code>	<code>V</code>	Schreibt einen Eintrag in die Map, aber nur, wenn kein Eintrag mit demselben Schlüssel existiert.	<pre>1 map.putIfAbsent("B", 20);</pre>
<code>remove(Object key)</code>	<code>V</code>	Entfernt einen Eintrag aus der Map.	<pre>1 Integer old = map.remove ("A");</pre>
<code>replace(K key, V value)</code>	<code>V</code>	Schreibt in einen Eintrag in der Map, nur wenn er existiert.	<pre>1 Integer old = map. replace("A", 100);</pre>
<code>replaceAll(BiFunction<? super K, ? super V, ? extends V> function)</code>	<code>void</code>	Ersetzt den Wert jedes Eintrags mit dem Ergebnis einer Operation.	<pre>1 map.replaceAll((k,v) -> v*2);</pre>
<code>size()</code>	<code>int</code>	Gibt die Anzahl der Einträge in der Map zurück.	<pre>1 int sz = map.size();</pre>
<code>values()</code>	<code>Collection<V></code>	Gibt eine Sammlung zurück, die alle Werte in der Map enthält.	<pre>1 Collection<Integer> vals = map.values();</pre>

10 Java Scanner-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>close()</code>	<code>void</code>	Schließt das Scanner-Objekt.	<pre> 1 Scanner sc = new Scanner (System.in); 2 sc.close(); </pre>
<code>delimiter()</code>	<code>Pattern</code>	Gibt das Trennzeichen zurück, das zum Trennen von Tokens verwendet wird.	<pre> Pattern p = sc.delimiter (); </pre>
<code>findInLine(String pattern)</code>	<code>String</code>	Gibt einen String zurück, der mit einem regulären Ausdruck in der aktuellen Zeile übereinstimmt.	<pre> String match = sc. findInLine("\\d+"); </pre>
<code>findWithinHorizon(String pattern, int horizon)</code>	<code>String</code>	Gibt einen String zurück, der mit einem regulären Ausdruck innerhalb einer angegebenen Anzahl von Zeichen übereinstimmt.	<pre> String m = sc. findWithinHorizon(" \\d+", 10); </pre>
<code>hasNext()</code>	<code>boolean</code>	Gibt <code>true</code> zurück, wenn ein weiteres Token gefunden werden kann.	<pre> boolean has = sc.hasNext (); </pre>
<code>hasNextBoolean()</code>	<code>boolean</code>	Gibt <code>true</code> zurück, wenn das nächste Token einen <code>boolean</code> -Wert darstellt.	<pre> boolean hb = sc. hasNextBoolean(); </pre>
<code>hasNextByte()</code>	<code>boolean</code>	Gibt <code>true</code> zurück, wenn das nächste Token einen <code>byte</code> -Wert darstellt.	<pre> boolean hb = sc. hasNextByte(); </pre>
<code>hasNextDouble()</code>	<code>boolean</code>	Gibt <code>true</code> zurück, wenn das nächste Token eine Zahl darstellt.	<pre> boolean hd = sc. hasNextDouble(); </pre>
<code>hasNextFloat()</code>	<code>boolean</code>	Gibt <code>true</code> zurück, wenn das nächste Token eine Zahl darstellt.	<pre> boolean hf = sc. hasNextFloat(); </pre>
<code>hasNextInt()</code>	<code>boolean</code>	Gibt <code>true</code> zurück, wenn das nächste Token einen <code>int</code> -Wert darstellt.	<pre> boolean hi = sc. hasNextInt(); </pre>
<code>hasNextLine()</code>	<code>boolean</code>	Gibt <code>true</code> zurück, wenn eine weitere Textzeile im Scanner verfügbar ist.	<pre> boolean hl = sc. hasNextLine(); </pre>
<code>hasNextLong()</code>	<code>boolean</code>	Gibt <code>true</code> zurück, wenn das nächste Token einen <code>long</code> -Wert darstellt.	<pre> boolean hl = sc. hasNextLong(); </pre>
<code>hasNextShort()</code>	<code>boolean</code>	Gibt <code>true</code> zurück, wenn das nächste Token einen <code>short</code> -Wert darstellt.	<pre> boolean hs = sc. hasNextShort(); </pre>

Fortsetzung auf nächster Seite...

Methoden	Rückgabebetyp	Beschreibung	Beispiel
locale()	Locale	Gibt die Lokalisierung des Scanners zurück.	<pre>Locale loc = sc.locale();</pre>
next()	String	Gibt das nächste Token im Scanner zurück.	<pre>String token = sc.next();</pre>
nextBoolean()	boolean	Gibt den boolean-Wert des nächsten Tokens im Scanner zurück.	<pre>boolean b = sc.nextBoolean();</pre>
nextByte()	byte	Gibt den byte-Wert des nächsten Tokens im Scanner zurück.	<pre>byte by = sc.nextByte();</pre>
nextDouble()	double	Gibt den double-Wert des nächsten Tokens im Scanner zurück.	<pre>double d = sc.nextDouble();</pre>
nextFloat()	float	Gibt den float-Wert des nächsten Tokens im Scanner zurück.	<pre>float f = sc.nextFloat();</pre>
nextInt()	int	Gibt den int-Wert des nächsten Tokens im Scanner zurück.	<pre>int i = sc.nextInt();</pre>
nextLine()	String	Gibt die nächste Textzeile im Scanner zurück.	<pre>String line = sc.nextLine();</pre>
nextLong()	long	Gibt den long-Wert des nächsten Tokens im Scanner zurück.	<pre>long l = sc.nextLong();</pre>
nextShort()	short	Gibt den short-Wert des nächsten Tokens im Scanner zurück.	<pre>short s = sc.nextShort();</pre>
radix()	int	Gibt die Basis des Scanners zurück.	<pre>int r = sc.radix();</pre>
reset()	Scanner	Setzt die Konfiguration des Scanners zurück.	<pre>sc.reset();</pre>
useDelimiter(String pattern)	Scanner	Setzt das Trennzeichen, das vom Scanner zum Trennen von Tokens verwendet wird.	<pre>sc.useDelimiter(",");</pre>
useLocale(Locale locale)	Scanner	Setzt die Lokalisierung, die vom Scanner verwendet wird.	<pre>sc.useLocale(Locale.GERMAN);</pre>
useRadix(int radix)	Scanner	Setzt die Basis, die vom Scanner verwendet wird.	<pre>sc.useRadix(16);</pre>

11 Java File-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>createNewFile()</code>	<code>boolean</code>	Erstellt eine neue, leere Datei, wenn sie noch nicht existiert.	<pre> 1 File f = new File("new. txt"); 2 boolean created = f. createNewFile(); </pre>
<code>delete()</code>	<code>boolean</code>	Löscht die Datei oder das Verzeichnis.	<pre> 1 boolean del = f.delete() ; </pre>
<code>exists()</code>	<code>boolean</code>	Prüft, ob die Datei oder das Verzeichnis existiert.	<pre> 1 boolean ex = f.exists(); </pre>
<code>canRead()</code>	<code>boolean</code>	Prüft, ob die Anwendung die Datei lesen kann.	<pre> 1 boolean r = f.canRead(); </pre>
<code>canWrite()</code>	<code>boolean</code>	Prüft, ob die Anwendung in die Datei schreiben kann.	<pre> 1 boolean w = f.canWrite() ; </pre>
<code>isFile()</code>	<code>boolean</code>	Gibt <code>true</code> zurück, wenn der Pfad eine Datei repräsentiert.	<pre> 1 boolean isF = f.isFile() ; </pre>
<code>isDirectory()</code>	<code>boolean</code>	Gibt <code>true</code> zurück, wenn der Pfad ein Verzeichnis repräsentiert.	<pre> 1 boolean isD = f. isDirectory(); </pre>
<code>length()</code>	<code>long</code>	Gibt die Größe der Datei in Bytes zurück.	<pre> 1 long len = f.length(); </pre>
<code>getName()</code>	<code>String</code>	Gibt den Namen der Datei oder des Verzeichnisses zurück.	<pre> 1 String name = f.getName (); </pre>
<code>getAbsolutePath()</code>	<code>String</code>	Gibt den absoluten Pfadnamen als String zurück.	<pre> 1 String path = f. getAbsolutePath(); </pre>
<code>getPath()</code>	<code>String</code>	Gibt den im Konstruktor angegebenen Pfad als String zurück (relativ oder absolut).	<pre> 1 String p = f.getPath(); </pre>
<code>getParent()</code>	<code>String</code>	Gibt den Namen des übergeordneten Verzeichnisses zurück oder <code>null</code> .	<pre> 1 String parent = f. getParent(); </pre>
<code>getCanonicalPath()</code>	<code>String</code>	Gibt den kanonischen (normalisierten) Pfad zurück. Wirft <code>IOException</code> .	<pre> 1 String canon = f. getCanonicalPath(); </pre>
<code>mkdir()</code>	<code>boolean</code>	Erstellt das Verzeichnis, falls es nicht existiert.	<pre> 1 File dir = new File(" ordner"); 2 boolean made = dir.mkdir (); </pre>

Fortsetzung auf nächster Seite...

Methoden	Rückgabebetyp	Beschreibung	Beispiel
<code>makedirs()</code>	<code>boolean</code>	Erstellt das Verzeichnis inklusive aller nicht existierenden Elternverzeichnisse.	<pre>boolean made = dir. makedirs();</pre>
<code>renameTo(File dest)</code>	<code>boolean</code>	Benennt die Datei oder das Verzeichnis um bzw. verschiebt es.	<pre>File neu = new File("neu .txt"); boolean renamed = f. renameTo(neu);</pre>
<code>list()</code>	<code>String[]</code>	Gibt ein Array mit den Namen der enthaltenen Dateien/Ordner zurück (nur bei Verzeichnis).	<pre>String[] inhalt = dir. list();</pre>
<code>listFiles()</code>	<code>File[]</code>	Gibt ein Array mit File-Objekten der enthaltenen Einträge zurück.	<pre>File[] files = dir. listFiles();</pre>
<code>setReadOnly()</code>	<code>boolean</code>	Macht die Datei schreibgeschützt.	<pre>boolean ro = f. setReadOnly();</pre>
<code>setLastModified(long time)</code>	<code>boolean</code>	Setzt den Zeitstempel der letzten Änderung (in Millisekunden seit 1970).	<pre>boolean set = f. setLastModified(System. currentTimeMillis());</pre>
<code>lastModified()</code>	<code>long</code>	Gibt den Zeitstempel der letzten Änderung zurück.	<pre>long mod = f. lastModified();</pre>
<code>isHidden()</code>	<code>boolean</code>	Prüft, ob die Datei als versteckt markiert ist.	<pre>boolean hidden = f. isHidden();</pre>

12 Java FileInputStream-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
available()	int	Anzahl lesbarer Bytes ohne Blockierung.	<pre>1 int n = fis.available();</pre>
close()	void	Schließt den Stream.	<pre>1 fis.close();</pre>
read()	int	Liest ein Byte (-1 bei EOF).	<pre>1 int b = fis.read();</pre>
read(byte[] b)	int	Liest in Array (Anzahl gelesener Bytes).	<pre>1 byte[] buf = new byte 2 [1024]; 2 int len = fis.read(buf);</pre>
read(byte[] b, int off, int len)	int	Liest Teilbereich des Arrays.	<pre>1 int len = fis.read(buf, 0, 10);</pre>
skip(long n)	long	Überspringt n Bytes.	<pre>1 long skipped = fis.skip (100);</pre>
getFD()	FileDescriptor	Gibt Dateideskriptor zurück.	<pre>1 FileDescriptor fd = fis. getFD();</pre>
getChannel()	FileChannel	Gibt zugehörigen Channel.	<pre>1 FileChannel ch = fis. getChannel();</pre>

13 Java FileOutputStream-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>close()</code>	<code>void</code>	Schließt den Stream.	<pre>1 fos.close();</pre>
<code>flush()</code>	<code>void</code>	Schreibt gepufferte Daten sofort.	<pre>1 fos.flush();</pre>
<code>write(int b)</code>	<code>void</code>	Schreibt ein Byte.	<pre>1 fos.write(65); // 'A'</pre>
<code>write(byte[] b)</code>	<code>void</code>	Schreibt gesamtes Array.	<pre>1 byte[] data = {1,2,3}; 2 fos.write(data);</pre>
<code>write(byte[] b, int off, int len)</code>	<code>void</code>	Schreibt Teilbereich.	<pre>1 fos.write(data, 1, 2); // 2,3</pre>
<code>getFD()</code>	<code>FileDescriptor</code>	Gibt Dateideskriptor.	<pre>1 FileDescriptor fd = fos. getFD();</pre>
<code>getChannel()</code>	<code>FileChannel</code>	Gibt zugehörigen Channel.	<pre>1 FileChannel ch = fos. getChannel();</pre>

14 Java BufferedReader-Methoden

Methode	Rückgabebetyp	Beschreibung	Beispiel
<code>close()</code>	<code>void</code>	Schließt den Reader.	<pre>1 br.close();</pre>
<code>mark(int limit)</code>	<code>void</code>	Markiert Position (max. limit).	<pre>1 br.mark(100);</pre>
<code>markSupported()</code>	<code>boolean</code>	Prüft Markierungsunterstützung.	<pre>1 boolean sup = br. markSupported();</pre>
<code>read()</code>	<code>int</code>	Liest ein Zeichen (-1 bei EOF).	<pre>1 int c = br.read();</pre>
<code>read(char[] cbuf)</code>	<code>int</code>	Liest in Array.	<pre>1 char[] buf = new char [10]; 2 int n = br.read(buf);</pre>
<code>readLine()</code>	<code>String</code>	Liest eine Zeile (ohne <code>\n</code>).	<pre>1 String line = br. readLine();</pre>
<code>ready()</code>	<code>boolean</code>	Prüft, ob lesebereit.	<pre>1 boolean r = br.ready();</pre>
<code>reset()</code>	<code>void</code>	Springt zur Markierung zurück.	<pre>1 br.reset();</pre>
<code>skip(long n)</code>	<code>long</code>	Überspringt <code>n</code> Zeichen.	<pre>1 long s = br.skip(5);</pre>
<code>lines()</code>	<code>Stream<String></code>	Gibt Stream aller Zeilen.	<pre>1 br.lines().forEach(System.out::println);</pre>

15 Java BufferedWriter-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>close()</code>	<code>void</code>	Schließt und leert.	<pre>1 bw.close();</pre>
<code>flush()</code>	<code>void</code>	Schreibt gepufferte Daten.	<pre>1 bw.flush();</pre>
<code>newLine()</code>	<code>void</code>	Schreibt Zeilentrenner.	<pre>1 bw.newLine();</pre>
<code>write(int c)</code>	<code>void</code>	Schreibt ein Zeichen.	<pre>1 bw.write('A');</pre>
<code>write(char[] cbuf)</code>	<code>void</code>	Schreibt Array.	<pre>1 char[] txt = {'H', 'i'}; 2 bw.write(txt);</pre>
<code>write(String s)</code>	<code>void</code>	Schreibt String.	<pre>1 bw.write("Hallo");</pre>
<code>append(CharSequence cs)</code>	<code>Writer</code>	Fügt Text an.	<pre>1 bw.append(" Welt");</pre>

16 Java Iterator-Methoden

Methode	Rückgabebetyp	Beschreibung	Beispiel
<code>hasNext()</code>	<code>boolean</code>	Prüft, ob weiteres Element.	<pre>if (it.hasNext())</pre>
<code>next()</code>	<code>T</code>	Gibt nächstes Element.	<pre>String s = it.next();</pre>
<code>remove()</code>	<code>void</code>	Entfernt letztes Element.	<pre>it.remove();</pre>

17 Java Collections-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>sort(List<T> list)</code>	<code>void</code>	Sortiert Liste aufsteigend.	<pre>Collections.sort(list);</pre>
<code>binarySearch(...)</code>	<code>int</code>	Binärsuche in sortierter Liste.	<pre>int i = Collections. binarySearch(list, 5);</pre>
<code>reverse(List<?> list)</code>	<code>void</code>	Kehrt Reihenfolge um.	<pre>Collections.reverse(list);</pre>
<code>shuffle(List<?> list)</code>	<code>void</code>	Zufällige Permutation.	<pre>Collections.shuffle(list);</pre>
<code>max(Collection<? extends T> c)</code>	<code>T</code>	Maximales Element.	<pre>Integer m = Collections. max(list);</pre>
<code>frequency(...)</code>	<code>int</code>	Anzahl Vorkommen.	<pre>int n = Collections. frequency(list, 3);</pre>
<code>unmodifiableList(...)</code>	<code>List<T></code>	Nur-Lese-Ansicht.	<pre>List<T> ro = Collections .unmodifiableList(list);</pre>

18 Java System-Methoden

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>currentTimeMillis()</code>	<code>long</code>	Zeit in ms seit 1970.	<pre>1 long t = System. currentTimeMillis() ;</pre>
<code>nanoTime()</code>	<code>long</code>	Hochpräziser Timer (ns).	<pre>1 long n = System.nanoTime ();</pre>
<code>exit(int status)</code>	<code>void</code>	Beendet JVM.	<pre>1 System.exit(0);</pre>
<code>gc()</code>	<code>void</code>	Schlägt GC vor.	<pre>1 System.gc();</pre>
<code>getProperty(String key)</code>	<code>String</code>	Liest Systemeigenschaft.	<pre>1 String v = System. getProperty("os. name");</pre>
<code>setProperty(...)</code>	<code>String</code>	Setzt Eigenschaft.	<pre>1 System.setProperty("x", "y");</pre>
<code>arraycopy(...)</code>	<code>void</code>	Kopiert Array.	<pre>1 System.arraycopy(src, 0, dst, 0, 5);</pre>
<code>lineSeparator()</code>	<code>String</code>	System-Zeilentrenner.	<pre>1 String sep = System. lineSeparator();</pre>

19 Java Fehler und Ausnahmen

Ausnahme/Fehler	Beschreibung	Beispiel
ArithmeticException	Division durch 0, etc.	<pre>1 int x = 1 / 0;</pre>
ArrayIndexOutOfBoundsException	Ungültiger Array-Index.	<pre>1 int[] a = {1}; a[5] = 0;</pre>
ClassNotFoundException	Klasse nicht gefunden.	<pre>1 Class.forName("X");</pre>
FileNotFoundException	Datei nicht gefunden.	<pre>1 new FileInputStream("x.txt");</pre>
IOException	Allgemeiner I/O-Fehler.	<pre>1 throw new IOException();</pre>
NullPointerException	Zugriff auf null.	<pre>1 String s = null; s.length();</pre>
NumberFormatException	Ungültige Zahl.	<pre>1 Integer.parseInt("abc");</pre>
IllegalArgumentException	Ungültiges Argument.	<pre>1 throw new IllegalArgumentException() ;</pre>

20 Java-Datum

Methode	Rückgabotyp	Beschreibung	Beispiel
adjustInto(Temporal temporal)	Temporal	Passt ein temporales Objekt an dieses Datum an.	<pre>1 LocalDateTime ldt = LocalDateTime.now(); 2 LocalDate d = LocalDate.now ().adjustInto(ldt);</pre>
atStartOfDay()	LocalDateTime	Datum + 00:00:00.	<pre>1 LocalDateTime start = LocalDate.now(). atStartOfDay();</pre>
atStartOfDay(ZoneId zone)	ZonedDateTime	Früheste gültige Zeit in Zeitzone.	<pre>1 ZonedDateTime zdt = LocalDate.now() 2 .atStartOfDay(ZoneId.of("Europe/Berlin"));</pre>
atTime(int hour, int minute)	LocalDateTime	Kombiniert mit Stunde/Minute.	<pre>1 LocalDateTime dt = LocalDate.now().atTime (14, 30);</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabebetyp	Beschreibung	Beispiel
<code>atTime(int hour, int minute, int second)</code>	<code>LocalDateTime</code>	Mit Sekunden.	<pre>1 LocalDateTime dt = LocalDateTime.now().atTime(14, 30, 0);</pre>
<code>atTime(int hour, int minute, int second, int nanoOfSecond)</code>	<code>LocalDateTime</code>	Mit Nanosekunden.	<pre>1 LocalDateTime dt = LocalDateTime.now() 2 .atTime(14, 30, 0, 500 _000_000);</pre>
<code>atTime(LocalTime time)</code>	<code>LocalDateTime</code>	Mit <code>LocalTime</code> .	<pre>1 LocalDateTime dt = LocalDateTime.now().atTime(LocalTime.NOON);</pre>
<code>atTime(OffsetTime time)</code>	<code>OffsetDateTime</code>	Mit <code>OffsetTime</code> .	<pre>1 OffsetTime ot = OffsetTime. of(LocalTime.NOON, ZoneOffset.UTC); 2 OffsetDateTime odt = LocalDateTime.now().atTime(ot);</pre>
<code>compareTo(ChronoLocalDate other)</code>	<code>int</code>	Vergleicht zwei Daten.	<pre>1 int cmp = LocalDateTime.now(). compareTo(LocalDate.of(2000,1,1));</pre>
<code>datesUntil(LocalDate endDateExclusive)</code>	<code>Stream<LocalDate></code>	Stream bis exklusiv.	<pre>1 long cnt = LocalDate.of((2025,1,1) 2 .datesUntil(LocalDate.of(2025,1,10)).count ()); // 9</pre>
<code>datesUntil(LocalDate endDateExclusive, Period step)</code>	<code>Stream<LocalDate></code>	Mit Schritt (z. B. monatlich).	<pre>1 LocalDateTime.now() 2 .datesUntil(LocalDate.now() ().plusYears(1), Period.ofMonths(2)) 3 .forEach(System.out:: println);</pre>
<code>equals(Object obj)</code>	<code>boolean</code>	Prüft Gleichheit.	<pre>1 boolean eq = LocalDateTime.now() ().equals(LocalDate.now() ()); // true</pre>
<code>format(DateTimeFormatter formatter)</code>	<code>String</code>	Formatiert mit Formatter.	<pre>1 String s = LocalDateTime.now() 2 .format(DateTimeFormatter. ofPattern("dd.MM. yyyy"));</pre>
<code>from(TemporalAccessor temporal)</code>	<code>static</code> <code>LocalDate</code>	Aus temporalem Objekt.	<pre>1 LocalDate d = LocalDate. from(LocalDateTime.now() ());</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
get(TemporalField field)	int	Wert eines Feldes.	<pre>int day = LocalDate.now().get(ChronoField.DAY_OF_MONTH);</pre>
getChronology()	IsoChronology	Gibt ISO-Chronologie.	<pre>IsoChronology c = LocalDate.now().getChronology();</pre>
getDayOfMonth()	int	Tag im Monat.	<pre>int tag = LocalDate.now().getDayOfMonth();</pre>
getDayOfWeek()	DayOfWeek	Wochentag.	<pre>DayOfWeek dow = LocalDate.now().getDayOfWeek();</pre>
getDayOfYear()	int	Tag im Jahr.	<pre>int doy = LocalDate.now().getDayOfYear();</pre>
getEra()	IsoEra	Ära (CE/BCE).	<pre>IsoEra era = LocalDate.now().getEra();</pre>
getLong(TemporalField field)	long	Feld als long.	<pre>long year = LocalDate.now().getLong(ChronoField.YEAR);</pre>
getMonth()	Month	Monat als Enum.	<pre>Month m = LocalDate.now().getMonth();</pre>
getMonthValue()	int	Monat (1–12).	<pre>int mv = LocalDate.now().getMonthValue();</pre>
getYear()	int	Jahr.	<pre>int y = LocalDate.now().getYear();</pre>
hashCode()	int	Hash-Code.	<pre>int h = LocalDate.now().hashCode();</pre>
isAfter(ChronoLocalDate other)	boolean	Ist nach?	<pre>boolean after = LocalDate.now().isAfter(LocalDate.of(2000,1,1));</pre>
isBefore(ChronoLocalDate other)	boolean	Ist vor?	<pre>boolean before = LocalDate.of(2000,1,1).isBefore(LocalDate.now());</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabebetyp	Beschreibung	Beispiel
<code>isEqual(ChronoLocalDate other)</code>	boolean	Ist gleich?	<pre>boolean eq = LocalDate.now().isEqual(LocalDate.now());</pre>
<code>isLeapYear()</code>	boolean	Schaltjahr?	<pre>boolean leap = LocalDate.of(2024,1,1).isLeapYear(); // true</pre>
<code>isSupported(TemporalField field)</code>	boolean	Feld unterstützt?	<pre>boolean sup = LocalDate.now().isSupported(ChronoField.HOUR_OF_DAY); // false</pre>
<code>isSupported(TemporalUnit unit)</code>	boolean	Einheit unterstützt?	<pre>boolean sup = LocalDate.now().isSupported(ChronoUnit.DAYS); // true</pre>
<code>lengthOfMonth()</code>	int	Tage im Monat.	<pre>int days = LocalDate.of(2025,2,1).lengthOfMonth(); // 28</pre>
<code>lengthOfYear()</code>	int	Tage im Jahr.	<pre>int days = LocalDate.of(2024,1,1).lengthOfYear(); // 366</pre>
<code>minus(long amount, TemporalUnit unit)</code>	LocalDate	Subtrahiert Einheit.	<pre>LocalDate past = LocalDate.now().minus(5, ChronoUnit.MONTHS);</pre>
<code>minus(TemporalAmount amount)</code>	LocalDate	Subtrahiert Period.	<pre>LocalDate past = LocalDate.now().minus(Period.ofYears(1));</pre>
<code>minusDays(long days)</code>	LocalDate	Subtrahiert Tage.	<pre>LocalDate ago = LocalDate.now().minusDays(7);</pre>
<code>minusMonths(long months)</code>	LocalDate	Subtrahiert Monate.	<pre>LocalDate m = LocalDate.now().minusMonths(3);</pre>
<code>minusWeeks(long weeks)</code>	LocalDate	Subtrahiert Wochen.	<pre>LocalDate w = LocalDate.now().minusWeeks(2);</pre>
<code>minusYears(long years)</code>	LocalDate	Subtrahiert Jahre.	<pre>LocalDate y = LocalDate.now().minusYears(10);</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
<code>now()</code>	static LocalDate	Aktuelles Datum.	<pre>1 LocalDate today = LocalDate .now();</pre>
<code>now(Clock clock)</code>	static LocalDate	Mit Clock.	<pre>1 LocalDate now = LocalDate. now(Clock.systemUTC());</pre>
<code>now(ZoneId zone)</code>	static LocalDate	In Zeitzone.	<pre>1 LocalDate berlin = LocalDate.now(ZoneId.of ("Europe/Berlin"));</pre>
<code>of(int year, int month, int day)</code>	static LocalDate	Aus Jahr/Monat/Tag.	<pre>1 LocalDate d = LocalDate.of (2025,12,25);</pre>
<code>of(int year, Month month, int day)</code>	static LocalDate	Mit Month.	<pre>1 LocalDate d = LocalDate.of (2025,Month.DECEMBER ,25);</pre>
<code>ofEpochDay(long epochDay)</code>	static LocalDate	Aus Epoch-Tag.	<pre>1 LocalDate d = LocalDate. ofEpochDay(0); // 1970-01-01</pre>
<code>ofInstant(Instant instant, ZoneId zone)</code>	static LocalDate	Aus Instant.	<pre>1 LocalDate d = LocalDate. ofInstant(Instant.now() , ZoneId.of("UTC"));</pre>
<code>ofYearDay(int year, int dayOfYear)</code>	static LocalDate	Aus Jahr + Tag im Jahr.	<pre>1 LocalDate d = LocalDate. ofYearDay(2025,100);</pre>
<code>parse(CharSequence text)</code>	static LocalDate	Aus YYYY-MM-DD.	<pre>1 LocalDate d = LocalDate. parse("2025-12-25");</pre>
<code>parse(CharSequence text, DateTimeFormatter fmt)</code>	static LocalDate	Mit Formatter.	<pre>1 DateTimeFormatter f = DateTimeFormatter. ofPattern("dd.MM.yyyy") ; 2 LocalDate d = LocalDate. parse("25.12.2025", f);</pre>
<code>plus(long amount, TemporalUnit unit)</code>	LocalDate	Addiert Einheit.	<pre>1 LocalDate future = LocalDate.now().plus(3, ChronoUnit.MONTHS);</pre>
<code>plus(TemporalAmount amount)</code>	LocalDate	Addiert Period.	<pre>1 LocalDate f = LocalDate.now ().plus(Period.ofDays (10));</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
plusDays(long days)	LocalDate	Addiert Tage.	<pre>1 LocalDate d = LocalDate.now() 2 .plusDays(5);</pre>
plusMonths(long months)	LocalDate	Addiert Monate.	<pre>1 LocalDate m = LocalDate.now() 2 .plusMonths(1);</pre>
plusWeeks(long weeks)	LocalDate	Addiert Wochen.	<pre>1 LocalDate w = LocalDate.now() 2 .plusWeeks(1);</pre>
plusYears(long years)	LocalDate	Addiert Jahre.	<pre>1 LocalDate y = LocalDate.now() 2 .plusYears(1);</pre>
query(TemporalQuery<R> query)	<R> R	Benutzerdefinierte Abfrage.	<pre>1 Year y = LocalDate.now() 2 .query(TemporalQueries.localDate().andThen(3 TemporalQueries::year));</pre>
range(TemporalField field)	ValueRange	Gültiger Wertebereich.	<pre>1 ValueRange r = LocalDate.now().range(ChronoField.DAY_OF_MONTH);</pre>
toEpochSecond(LocalTime time, ZoneOffset offset)	long	Sekunden seit 1970.	<pre>1 long sec = LocalDate.now().toEpochSecond(LocalTime.NOON, ZoneOffset.UTC);</pre>
toString()	String	YYYY-MM-DD.	<pre>1 String s = LocalDate.now().toString();</pre>
until(ChronoLocalDate endExclusive)	Period	Period zwischen Daten.	<pre>1 Period p = LocalDate.of(2000,1,1).until(LocalDate.now());</pre>
until(Temporal endExclusive, TemporalUnit unit)	long	Differenz in Einheit.	<pre>1 long days = LocalDate.now().until(LocalDate.of(2025,12,25), ChronoUnit.DAYS);</pre>
with(TemporalAdjuster adjuster)	LocalDate	Mit Adjuster.	<pre>1 LocalDate first = LocalDate.now().with(TemporalAdjusters.firstDayOfMonth());</pre>
with(TemporalField field, long value)	LocalDate	Setzt Feld.	<pre>1 LocalDate d = LocalDate.now().with(ChronoField.DAY_OF_MONTH, 15);</pre>

Fortsetzung auf nächster Seite...

Methode	Rückgabotyp	Beschreibung	Beispiel
withDayOfMonth(int dayOfMonth)	LocalDate	Ändert Tag im Monat.	<pre>1 LocalDate d = LocalDate.now 2 ().withDayOfMonth(1);</pre>
withDayOfYear(int dayOfYear)	LocalDate	Ändert Tag im Jahr.	<pre>1 LocalDate d = LocalDate.now 2 ().withDayOfYear(200);</pre>
withMonth(int month)	LocalDate	Ändert Monat.	<pre>1 LocalDate d = LocalDate.now 2 ().withMonth(12);</pre>
withYear(int year)	LocalDate	Ändert Jahr.	<pre>1 LocalDate d = LocalDate.now 2 ().withYear(2030);</pre>

21 Java-Typenumwandlungen

Umwandlung	Typen	Beschreibung	Beispiel
Widening Conversion	byte -> short -> int -> long -> float -> double	Automatische Typumwandlung von einem kleineren in einen größeren numerischen Typ.	<pre>1 int i = 10; 2 double d = i; // 10.0</pre>
Narrowing Conversion	double -> float -> long -> int -> short -> byte	Explizite Typumwandlung von einem größeren in einen kleineren numerischen Typ.	<pre>1 double d = 9.7; 2 int i = (int) d; // 9</pre>
Primitive -> Wrapper	int -> Integer usw.	Autoboxing: Ein primitiver Wert wird automatisch in ein Wrapper-Objekt umgewandelt.	<pre>1 Integer x = 5; // Autoboxing</pre>
Wrapper -> Primitive	Integer -> int usw.	Unboxing: Ein Wrapper-Objekt wird automatisch in einen primitiven Wert umgewandelt.	<pre>1 Integer x = 5; 2 int y = x; // Unboxing</pre>
String -> int	String, int	Umwandlung eines Strings in eine Zahl mit parseInt.	<pre>1 String s = "123"; 2 int i = Integer.parseInt (s);</pre>
String -> double	String, double	Umwandlung eines Strings in eine Kommazahl mit parseDouble.	<pre>1 String s = "3.14"; 2 double d = Double. parseDouble(s);</pre>
Zahl -> String	int, double,... -> String	Umwandlung einer Zahl in einen String über String.valueOf oder Verkettung.	<pre>1 int i = 42; 2 String s = String. valueOf(i);</pre>
char -> int	char -> int	Ein Zeichen kann als Unicode-Wert interpretiert werden.	<pre>1 char c = 'A'; 2 int code = c; // 65</pre>

Fortsetzung auf nächster Seite...

Umwandlung	Typen	Beschreibung	Beispiel
int -> char	int -> char	Ein numerischer Unicode-Wert kann in ein Zeichen umgewandelt werden.	<pre> 1 int code = 65; 2 char c = (char) code; // 'A' </pre>
String -> char[]	String -> char[]	Ein String kann mit toCharArray() in ein Zeichenarray umgewandelt werden.	<pre> 1 String s = "Java"; 2 char[] arr = s. toCharArray(); </pre>
char[] -> String	char[] -> String	Ein Zeichenarray kann mit dem Konstruktor oder String.valueOf in einen String umgewandelt werden.	<pre> 1 char[] arr = {'J','a','v','e'}; 2 String s = new String(arr); </pre>
Object -> String	Object -> String	Jedes Objekt kann über toString() oder String.valueOf als String dargestellt werden.	<pre> 1 Object o = 123; 2 String s = String. valueOf(o); </pre>
String -> boolean	String -> boolean	Ein String wird nicht direkt in boolean umgewandelt; meist wird verglichen oder geparkt, z. B. mit Boolean.parseBoolean.	<pre> 1 boolean b = Boolean. parseBoolean("true"); </pre>
boolean -> String	boolean -> String	Ein Boolean kann mit String.valueOf in einen String umgewandelt werden.	<pre> 1 boolean b = true; 2 String s = String. valueOf(b); </pre>
Casting bei Referenztypen	Superklasse -> Unterklasse	Downcasting ist nur mit explizitem Cast möglich und nur dann sicher, wenn das Objekt wirklich vom Zieltyp ist.	<pre> 1 Object o = "Hallo"; 2 String s = (String) o; </pre>

22 Java-Parsen

Ausdruck	Rückgabotyp	Beschreibung	Beispiel
Integer.parseInt(String s)	int	Wandelt einen String mit einer ganzen Zahl in einen int um.	<pre> 1 String s = "123"; 2 int n = Integer.parseInt (s); </pre>
Integer.parseInt(String s, int radix)	int	Wandelt einen String in einen int um, wobei die Basis radix angegeben wird.	<pre> 1 String s = "1010"; 2 int n = Integer.parseInt (s, 2); // 10 </pre>
Double.parseDouble(String s)	double	Wandelt einen String mit einer Kommazahl in einen double um.	<pre> 1 String s = "3.14"; 2 double x = Double. parseDouble(s); </pre>

Fortsetzung auf nächster Seite...

Ausdruck	Rückgabebetyp	Beschreibung	Beispiel
<code>Float.parseFloat(String s)</code>	<code>float</code>	Wandelt einen String mit einer Kommazahl in einen <code>float</code> um.	<pre> 1 String s = "2.5"; 2 float x = Float. parseFloat(s); </pre>
<code>Long.parseLong(String s)</code>	<code>long</code>	Wandelt einen String mit einer ganzen Zahl in einen <code>long</code> um.	<pre> 1 String s = "999999999"; 2 long n = Long.parseLong(s); </pre>
<code>Short.parseShort(String s)</code>	<code>short</code>	Wandelt einen String mit einer ganzen Zahl in einen <code>short</code> um.	<pre> 1 String s = "123"; 2 short n = Short. parseShort(s); </pre>
<code>Byte.parseByte(String s)</code>	<code>byte</code>	Wandelt einen String mit einer ganzen Zahl in einen <code>byte</code> um.	<pre> 1 String s = "7"; 2 byte n = Byte.parseByte(s); </pre>
<code>Boolean.parseBoolean(String s)</code>	<code>boolean</code>	Wandelt den String <code>"true"</code> in <code>true</code> um; alle anderen Werte ergeben <code>false</code> .	<pre> 1 boolean b1 = Boolean. parseBoolean("true"); 2 boolean b2 = Boolean. parseBoolean("false"); </pre>
<code>Character.getNumericValue(char c)</code>	<code>int</code>	Wandelt ein einzelnes Ziffernzeichen in den numerischen Wert um.	<pre> 1 char c = '7'; 2 int n = Character. getNumericValue(c); </pre>
<code>char - '0'</code>	<code>int</code>	Sehr kurze Möglichkeit, eine einzelne Ziffer aus einem Zeichen zu machen.	<pre> 1 char c = '7'; 2 int n = c - '0'; </pre>
<code>String.valueOf(...)</code>	<code>String</code>	Wandelt Werte in einen String um, oft als Zwischenschritt beim Parsen.	<pre> 1 char c = '7'; 2 String s = String. valueOf(c); 3 int n = Integer.parseInt(s); </pre>
<code>Integer.valueOf(String s)</code>	<code>Integer</code>	Wandelt einen String in ein <code>Integer</code> -Objekt um.	<pre> 1 String s = "123"; 2 Integer n = Integer. valueOf(s); </pre>
<code>Double.valueOf(String s)</code>	<code>Double</code>	Wandelt einen String in ein <code>Double</code> -Objekt um.	<pre> 1 String s = "3.14"; 2 Double x = Double. valueOf(s); </pre>

Java: Weitere Typumwandlungen

Umwandlung	Rückgabotyp	Beschreibung	Beispiel
<code>String.valueOf(int)</code>	String	Wandelt einen <code>int</code> in einen String um.	<pre> 1 <code>int</code> n = 42; 2 <code>String</code> s = <code>String</code>. <code>valueOf</code>(n); </pre>
<code>String.valueOf(double)</code>	String	Wandelt einen <code>double</code> in einen String um.	<pre> 1 <code>double</code> x = 3.14; 2 <code>String</code> s = <code>String</code>. <code>valueOf</code>(x); </pre>
<code>String.valueOf(boolean)</code>	String	Wandelt einen <code>boolean</code> in einen String um.	<pre> 1 <code>boolean</code> b = <code>true</code>; 2 <code>String</code> s = <code>String</code>. <code>valueOf</code>(b); </pre>
<code>Integer.valueOf(String)</code>	Integer	Wandelt einen String in ein <code>Integer</code> -Objekt um.	<pre> 1 <code>String</code> s = <code>"123"</code>; 2 <code>Integer</code> n = <code>Integer</code>. <code>valueOf</code>(s); </pre>
<code>Double.valueOf(String)</code>	Double	Wandelt einen String in ein <code>Double</code> -Objekt um.	<pre> 1 <code>String</code> s = <code>"3.14"</code>; 2 <code>Double</code> x = <code>Double</code>. <code>valueOf</code>(s); </pre>
<code>Boolean.valueOf(String)</code>	Boolean	Wandelt einen String in ein <code>Boolean</code> -Objekt um.	<pre> 1 <code>String</code> s = <code>"true"</code>; 2 <code>Boolean</code> b = <code>Boolean</code>. <code>valueOf</code>(s); </pre>
<code>Character.valueOf(char)</code>	Character	Wandelt ein <code>char</code> in ein <code>Character</code> -Objekt um.	<pre> 1 <code>char</code> c = <code>'A'</code>; 2 <code>Character</code> ch = <code>Character</code> <code>.valueOf</code>(c); </pre>
Autoboxing	Primitive -> Wrapper	Automatische Umwandlung eines primitiven Typs in das passende Wrapper-Objekt.	<pre> 1 <code>int</code> n = 5; 2 <code>Integer</code> x = n; </pre>
Unboxing	Wrapper -> Primitive	Automatische Umwandlung eines Wrapper-Objekts in den primitiven Typ.	<pre> 1 <code>Integer</code> x = 5; 2 <code>int</code> n = x; </pre>
Widening	klein -> groß	Automatische Erweiterung eines numerischen Typs ohne Cast.	<pre> 1 <code>int</code> n = 10; 2 <code>double</code> d = n; </pre>
Narrowing	groß -> klein	Explizite Verengung eines numerischen Typs mit Cast.	<pre> 1 <code>double</code> d = 9.8; 2 <code>int</code> n = (<code>int</code>) d; </pre>
<code>char[] -> String</code>	String	Wandelt ein Zeichenarray in einen String um.	<pre> 1 <code>char</code>[] arr = {<code>'J'</code>, <code>'a'</code>, <code>'v'</code> <code>'</code>, <code>'a'</code><code>'</code>}; 2 <code>String</code> s = <code>new String</code>(arr); </pre>

Fortsetzung auf nächster Seite...

Umwandlung	Rückgabotyp	Beschreibung	Beispiel
String -> char[]	char[]	Wandelt einen String in ein Zeichenarray um.	<pre> 1 String s = "Java"; 2 char[] arr = s.toCharArray(); </pre>
String -> byte[]	byte[]	Wandelt einen String in ein Byte-Array um, z. B. mit einer Zeichencodierung.	<pre> 1 String s = "Hi"; 2 byte[] data = s.getBytes(); </pre>
byte[] -> String	String	Wandelt ein Byte-Array in einen String um.	<pre> 1 byte[] data = {72, 105}; 2 String s = new String(data); </pre>
String -> char	char	Ein einzelnes Zeichen wird mit charAt aus dem String gelesen.	<pre> 1 String s = "Java"; 2 char c = s.charAt(0); </pre>
char -> String	String	Ein einzelnes Zeichen kann mit String.valueOf in einen String umgewandelt werden.	<pre> 1 char c = 'A'; 2 String s = String.valueOf(c); </pre>
Object -> String	String	Jedes Objekt kann über toString() oder String.valueOf als String dargestellt werden.	<pre> 1 Object o = 123; 2 String s = String.valueOf(o); </pre>
String -> BigInteger	BigInteger	Wandelt einen String in eine große Ganzzahl um.	<pre> 1 BigInteger n = new BigInteger(" 12345678901234567890 "); </pre>
String -> BigDecimal	BigDecimal	Wandelt einen String in eine Dezimalzahl mit hoher Genauigkeit um.	<pre> 1 BigDecimal x = new BigDecimal("3.14159 "); </pre>

23 Java Random-Klasse

Methode	Rückgabotyp	Beschreibung	Beispiel
nextInt()	int	Erzeugt eine zufällige Ganzzahl.	<pre> 1 Random r = new Random(); 2 int x = r.nextInt(); </pre>
nextInt(int bound)	int	Zufallszahl zwischen 0 (inkl.) und bound (exkl.).	<pre> 1 Random r = new Random(); 2 int x = r.nextInt(10); // 0-9 </pre>
nextDouble()	double	Zufallszahl zwischen 0.0 und 1.0.	<pre> 1 Random r = new Random(); 2 double d = r.nextDouble(); </pre>

Methode	Rückgabotyp	Beschreibung	Beispiel
nextBoolean()	boolean	Liefert zufällig true oder false.	<pre> 1 Random r = new Random(); 2 boolean b = r. nextBoolean(); </pre>
nextLong()	long	Erzeugt eine zufällige long-Zahl.	<pre> 1 Random r = new Random(); 2 long l = r.nextLong(); </pre>
setSeed(long seed)	void	Setzt einen festen Startwert für reproduzierbare Zufallszahlen.	<pre> 1 Random r = new Random(); 2 r.setSeed(42); </pre>
Collections.shuffle(List<? list)	void	Mischt die Reihenfolge einer Liste zufällig.	<pre> 1 List<Integer> nums = 2 Arrays.asList(1,2,3,4); 3 4 Collections.shuffle(nums); </pre>
shuffle ohne Zurücklegen	int[]	Zufällige Reihenfolge ohne doppelte Ziehungen.	<pre> 1 List<Integer> zahlen = 2 new ArrayList<>(); 3 4 for(int i=1;i<=10;i++){ 5 zahlen.add(i); 6 } 7 8 Collections.shuffle(9 zahlen); 10 11 System.out.println(zahlen); </pre>
Zufallszahl im Bereich	int	Zufallszahl zwischen zwei Werten.	<pre> 1 Random r = new Random(); 2 3 int min = 5; 4 int max = 15; 5 6 int x = 7 r.nextInt(max-min+1) 8 + min; </pre>
Math.random()	double	Alternative zu Random.	<pre> 1 double x = Math.random() 2 ; 3 4 int zahl = (int)(Math.random()*10); </pre>

Java Random-Klasse

Methode	Rückgabotyp	Beschreibung	Beispiel
new Random()	Random	Erstellt ein neues Zufallsobjekt. Ohne Seed entstehen bei jedem Programmstart andere Zufallszahlen.	<pre> 1 Random rand = new Random (); </pre>

Methode	Rückgabotyp	Beschreibung	Beispiel
Random(long seed)	Random	Erstellt einen Zufallsgenerator mit festem Startwert (Seed). Damit entstehen immer dieselben Zufallszahlenfolgen. Praktisch für Tests.	<pre> 1 Random rand = 2 new Random(42); </pre>
nextInt()	int	Liefert eine zufällige Ganzzahl aus dem gesamten int-Bereich. Werte können positiv oder negativ sein.	<pre> 1 Random rand = new Random 2 (); 3 int x = rand.nextInt(); </pre>
nextInt(int bound)	int	Erzeugt eine Zufallszahl von 0 (inkl.) bis bound (exkl.). Sehr häufig verwendet.	<pre> 1 Random rand = new Random 2 (); 3 int zahl = 4 rand.nextInt(10); 5 6 // Werte: 0-9 </pre>
nextInt(max-min+1)+min	int	Erzeugt eine Zufallszahl in einem bestimmten Bereich. Beide Grenzen sind inklusive.	<pre> 1 Random rand = new Random 2 (); 3 int min = 5; 4 int max = 15; 5 6 int zahl = 7 rand.nextInt(8 max-min+1 9) + min; 10 11 // Werte: 5-15 </pre>
nextDouble()	double	Liefert eine Gleitkommazahl zwischen 0.0 (inkl.) und 1.0 (exkl.).	<pre> 1 Random rand = new Random 2 (); 3 double d = 4 rand.nextDouble(); </pre>
nextFloat()	float	Funktioniert wie nextDouble(), jedoch mit dem Datentyp float.	<pre> 1 Random rand = new Random 2 (); 3 float f = 4 rand.nextFloat(); </pre>
nextBoolean()	boolean	Gibt zufällig true oder false zurück.	<pre> 1 Random rand = new Random 2 (); 3 boolean b = 4 rand.nextBoolean(); </pre>
nextLong()	long	Liefert eine zufällige Zahl vom Typ long.	<pre> 1 Random rand = new Random 2 (); 3 long l = 4 rand.nextLong(); </pre>

Methode	Rückgabotyp	Beschreibung	Beispiel
setSeed(long seed)	void	Setzt den Seed eines bestehenden Zufallsgenerators neu. Dadurch wird die Zufallsfolge reproduzierbar.	<pre> 1 Random rand = new Random 2 (); 3 rand.setSeed(1234); 4 5 System.out.println(6 rand.nextInt(10) 7); </pre>
Math.random()	double	Alternative zur Klasse Random. Liefert Werte zwischen 0.0 und 1.0.	<pre> 1 double x = 2 Math.random(); </pre>
(int)(Math.random()*10)	int	Erzeugt eine Ganzzahl zwischen 0 und 9.	<pre> 1 int zahl = 2 (int)(3 Math.random()*10 4); </pre>
Collections.shuffle(List)	void	Mischt die Reihenfolge aller Elemente einer Liste zufällig. Ideal für Karten- oder Quizspiele.	<pre> 1 List<Integer> zahlen = 2 Arrays.asList(3 1,2,3,4,5 4); 5 6 Collections.shuffle(7 zahlen 8); 9 10 System.out.println(11 zahlen 12); </pre>
Ziehen ohne Zurücklegen	int	Durch Mischen einer Liste kann jedes Element genau einmal zufällig gezogen werden.	<pre> 1 List<Integer> zahlen = 2 new ArrayList<>(); 3 4 for(int i=1;i<=10;i++){ 5 zahlen.add(i); 6 } 7 8 Collections.shuffle(9 zahlen 10); 11 12 int gezogen = 13 zahlen.remove(0); 14 15 System.out.println(16 gezogen 17); </pre>

Methode	Rückgabebetyp	Beschreibung	Beispiel
Zufälliges Array-Element	String	Wählt zufällig ein Element aus einem Array aus.	<pre> 1 String[] namen = { 2 "Anna", 3 "Tom", 4 "Lisa" 5 }; 6 7 Random rand = 8 new Random(); 9 10 String name = 11 namen[12 rand.nextInt(13 namen.length 14) 15]; </pre>
Mehrere Zufallszahlen	void	Beispiel zum Erzeugen mehrerer Zufallszahlen in einer Schleife.	<pre> 1 Random rand = 2 new Random(); 3 4 for(int i=0;i<5;i++){ 5 6 int zahl = 7 rand.nextInt(100); 8 9 System.out.println(10 zahl 11); 12 } </pre>